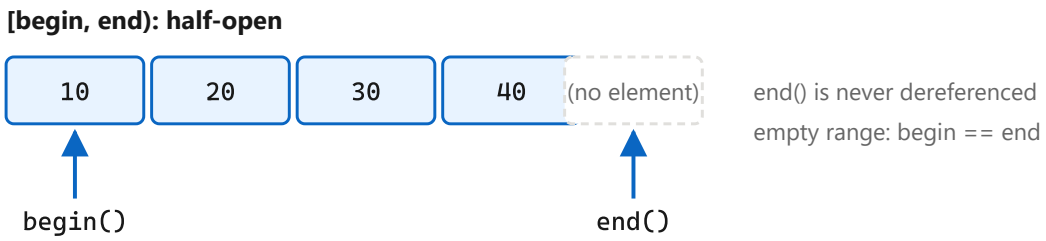
Iterators

Iterators are generalised pointers: they connect containers to algorithms. The half-open range, the iterator categories, inserters and invalidation, plus span and string_view as non-owning views. Assumes STL Containers.

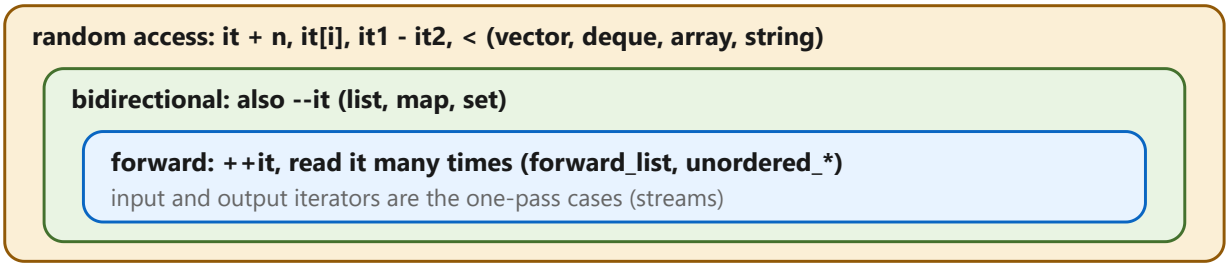
Iterators

An iterator acts like a pointer: `*it`, `++it`, `it == end`. `end()` is **one past the last**; ranges are half-open `[begin, end)`.



A loop runs while it != end; end() is a marker, not an element.

```
std::vector<int> v{1, 2, 3};
// use ≠ not <, it works for every container
for (auto it = v.begin(); it ≠ v.end(); ++it) std::cout << *it;
// range-for is exactly this loop in disguise
for (int x : v) std::cout << x;
// reverse iterators
for (auto it = v.rbegin(); it ≠ v.rend(); ++it) { }
// cbegin/cend: read-only, *it cannot be modified
for (auto it = v.cbegin(); it ≠ v.cend(); ++it) { }
// advance without assuming random access
auto it2 = std::next(v.begin(), 2);
```



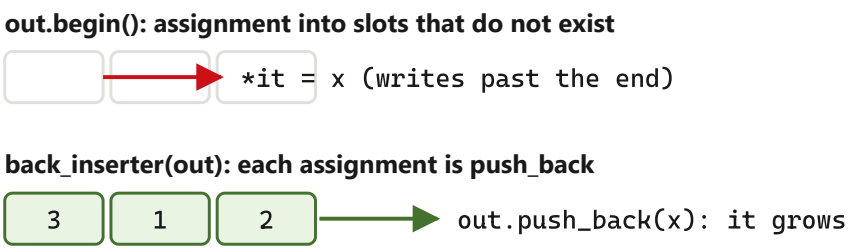
An algorithm asks for the weakest category it can work with; std::sort needs the biggest one.

CATEGORY	CAN DO	CONTAINERS
Forward	`++`, read (multi-pass)	`forward_list`, unordered containers
Bidirectional	also `--`	`list`, `map`, `set`
Random access	also `it + n`, `it[i]`, `it1 - it2`, `<`	`vector`, `deque`, `array`, `string`

So `std::sort` needs random access and fails on `std::list` (use `list.sort()`).

Inserters and iterator adaptors

An algorithm like `std::copy` **assigns** through its output iterator, so the destination must already be big enough. An **inserter** turns each assignment into an insertion instead:



An inserter turns assignment into insertion, so the destination does not need to be big enough first.

ADAPTOR	EACH ASSIGNMENT DOES
`std::back_inserter(c)`	`c.push_back(x)`
`std::front_inserter(c)`	`c.push_front(x)` (so the order reverses; needs `deque` / `list`)
`std::inserter(c, pos)`	`c.insert(pos, x)` (works for `set` and `map`)

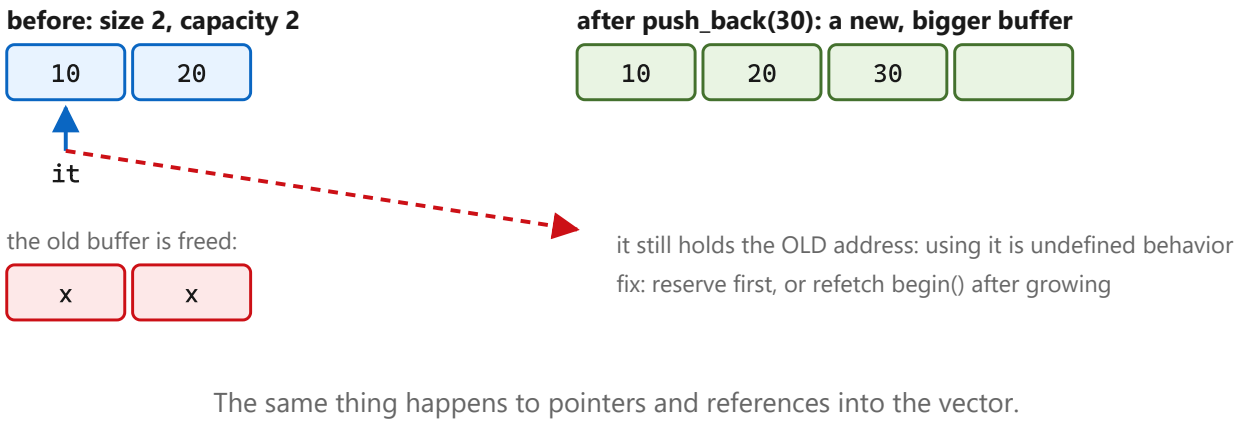
```cpp
std::vector<int> src{3, 1, 2}, out;
// out is {3, 1, 2}: it grew
std::copy(src.begin(), src.end(), std::back_inserter(out));
// std::copy(src.begin(), src.end(), out.begin()); // BUG if out is empty:
// writes past the end

std::set<int> sorted;
std::copy(src.begin(), src.end(), std::inserter(sorted, sorted.begin()));
```

`std::reverse_iterator` (what `rbegin()` returns) and `std::move_iterator` (moves instead of copying) are the other two adaptors worth knowing.

Iterator invalidation

Changing a container can **invalidate** iterators, pointers and references; using one is UB.



The same thing happens to pointers and references into the vector.

CONTAINER	OPERATION	WHAT IS INVALIDATED
`vector`	`push_back`, `insert`	**all** if it reallocated; otherwise those at or after the insertion point
`vector`	`erase`	the erased element and everything after it
`deque`	insert/erase in the middle	everything; push at either end invalidates iterators but not references
`list`, `map`, `set`	insert	nothing
`list`, `map`, `set`	erase	only the erased element
`unordered_*`	insert that triggers a rehash	all iterators (references stay valid)

▼ SPOT THE BUG Erasing inside a loop

```
std::vector<int> v{1, 2, 3, 4};
for (auto it = v.begin(); it ≠ v.end(); ++it)
    if (*it % 2 == 0) v.erase(it);          // BUG
```

`erase` invalidates `it`, and the loop then does `++it` on a dead iterator. The correct idiom uses the iterator `erase` returns:

```
for (auto it = v.begin(); it ≠ v.end(); ) {
    // erase returns the next valid iterator
    if (*it % 2 == 0) it = v.erase(it);
    else ++it;
}
// or, in C++20:
std::erase_if(v, [](int x) { return x % 2 == 0; });
```

`span` and `string_view`: non-owning views

A **view** is a pointer plus a length: it looks at elements that someone else owns. `std::span<T>` (C++20) views any contiguous run of `T`; `std::string_view` (C++17) is the same idea for characters.

```
long sum(std::span<const int> xs) { long t = 0; for (int x : xs) t += x;
return t; }

std::vector<int> vec{1, 2, 3, 4, 5};
int arr[3] = {10, 20, 30};
// 15: a vector converts to a span with no copy
sum(vec);
// 60: so does a C array, and its size is remembered
sum(arr);
// 9: elements 2, 3, 4: a slice is just a new pointer and length
sum(std::span(vec).subspan(1, 3));
sum(std::span(vec).first(2));          // 3
```

- A span is two words (compiled: 16 bytes on a 64-bit machine). Pass it **by value**.
- `span<const int>` is read-only; `span<int>` lets the function modify the elements in place. It still cannot grow or shrink the container.
- It replaces the `(T* data, size_t n)` pair. Overflows are easier to spot, and a function written against `span` accepts a vector, an array, a `std::array` or a part of any of them.
- **Dangling.** A view must not outlive what it looks at. A `span` of a vector that then reallocates (`push_back`), or a `string_view` of a temporary string, points at freed memory. Use views for parameters and short-lived local use, not for members that live longer than the data.

> RULE OF THUMB Take a view to *read or modify* existing elements; take or return the owning type (`vector`, `string`) to *keep* them.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Iterators	C++ Primer	3.4 Introducing iterators	135 to 141
	C++ Primer	10.4 Revisiting iterators	430 to 438
	C++ Primer	10.5 Structure of generic algorithms	439 to 443
	Programming: Principles and Practice	19.1 to 19.3, 19.6 Containers, sequences and iterators, linked lists, vector/list/string	559 to 572 and 585 to 586